# Subject : LP-3

- Section : Machine Learning
- Experiment : ML - 1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error
```

```
# 1. Upload dataset
uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))

print("Dataset Shape:", df.shape)
print(df.head())
```

```
Choose Files uber.csv
uber.csv(text/csv) - 23458139 bytes, last modified: 21/10/2026 - 100% done
Saving uber.csv to uber (1).csv
Dataset Shape: (200000, 9)
   Unnamed: 0                            key  fare_amount  \
0    24238194    2015-05-07 19:52:06.0000003          7.5
1    27835199    2009-07-17 20:04:56.0000002          7.7
2    44984355   2009-08-24 21:45:00.00000061         12.9
3    25894730    2009-06-26 08:22:21.0000001          5.3
4    17610152  2014-08-28 17:47:00.000000188         16.0

           pickup_datetime  pickup_longitude  pickup_latitude  \
0  2015-05-07 19:52:06 UTC        -73.999817        40.738354
1  2009-07-17 20:04:56 UTC        -73.994355        40.728225
2  2009-08-24 21:45:00 UTC        -74.005043        40.740770
3  2009-06-26 08:22:21 UTC        -73.976124        40.790844
4  2014-08-28 17:47:00 UTC        -73.925023        40.744085

   dropoff_longitude  dropoff_latitude  passenger_count
0         -73.999512         40.723217                1
1         -73.994710         40.750325                1
2         -73.962565         40.772647                1
3         -73.965316         40.803349                3
4         -73.973082         40.761247                5
```

```
# 2. Preprocess dataset
df = df.dropna()

df = df[
    (df["fare_amount"] > 0) &
    (df["fare_amount"] <= 500) &
    (df["passenger_count"] > 0)
]

# Remove invalid coordinates
df = df[
    df["pickup_longitude"].between(-180, 180) &
    df["dropoff_longitude"].between(-180, 180) &
    df["pickup_latitude"].between(-90, 90) &
    df["dropoff_latitude"].between(-90, 90)
]

print("Shape after cleaning:", df.shape)
```

```
Shape after cleaning: (199257, 9)
```

```
# 3. Identify outliers using IQR
Q1 = df["fare_amount"].quantile(0.25)
Q3 = df["fare_amount"].quantile(0.75)
IQR = Q3 - Q1

df = df[
    df["fare_amount"].between(
        Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    )
]

print("Shape after outlier removal:", df.shape)
```

```
Shape after outlier removal: (182137, 9)
```

```
# 4. Check correlation
features = [
    "pickup_longitude", "pickup_latitude",
    "dropoff_longitude", "dropoff_latitude",
    "passenger_count"
]

print("\nCorrelation Matrix:")
print(df[features + ["fare_amount"]].corr().round(2))
```

```
Correlation Matrix:
                   pickup_longitude  pickup_latitude  dropoff_longitude  \
pickup_longitude               1.00            -0.98               0.96
pickup_latitude               -0.98             1.00              -0.94
dropoff_longitude              0.96            -0.94               1.00
dropoff_latitude              -0.94             0.96              -0.98
passenger_count               -0.00            -0.00              -0.00
fare_amount                    0.00            -0.00              -0.00

                   dropoff_latitude  passenger_count  fare_amount
pickup_longitude              -0.94            -0.00         0.00
pickup_latitude                0.96            -0.00        -0.00
dropoff_longitude             -0.98            -0.00        -0.00
dropoff_latitude               1.00            -0.00         0.00
passenger_count               -0.00             1.00         0.01
fare_amount                    0.00             0.01         1.00
```

```
# 5. Split dataset
X = df[features]
y = df["fare_amount"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# 6. Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
pred_lr = lr.predict(X_test)
```

```
# 7. Random Forest Regression
rf = RandomForestRegressor(
    n_estimators=50, random_state=42
)
rf.fit(X_train, y_train)
pred_rf = rf.predict(X_test)
```

```
# 8. Evaluate models
print("\nLinear Regression")
print("R2 Score:", round(r2_score(y_test, pred_lr), 3))
print("RMSE:", round(np.sqrt(
    mean_squared_error(y_test, pred_lr)
), 3))

print("\nRandom Forest Regression")
print("R2 Score:", round(r2_score(y_test, pred_rf), 3))
print("RMSE:", round(np.sqrt(
    mean_squared_error(y_test, pred_rf)
), 3))
```

```
Linear Regression
R2 Score: 0.0
RMSE: 4.137

Random Forest Regression
R2 Score: 0.706
RMSE: 2.245
```